최종 배포 실험 리포트: smile_final.yaml + MICE 치아 템플릿 교체

> 코드가 어디서 분기되고 파라미터가 어디로 흘러가는지(디버깅용)는 core_architecture.md 참고.

> **추가 조사(중요, 아래 0장) 결과로 3장의 결론이 수정됐다.** "MICE 템플릿이 거의 효과 없다"는 우리가 쓰던 **"전체 생성 후 RGB 합성"** 방식 전반의 한계였고, diffusers에 이미 있는 **진짜 inpainting 파이프라인(`Flux2KleinInpaintPipeline`, `mask_image`)으로 바꾸면 같은 속도대에서 훨씬 잘 된다.** 자세한 내용은 0장.

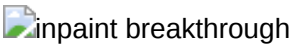
0. 추가 조사: 진짜 원인은 "합성(composite) 방식" 자체였다

사용자가 예전에 "MICE+템플릿+직접 프롬프트"로 UI 앱에서 좋은 결과를 얻은 적이 있다고 해서, 그 프롬프트(`image_1: Preserve facial identity... / mask_1: Replace the masked mouth region with a full set of clean, natural, well-aligned white teeth...`)를 그대로 재현해 봤다. 핵심은 **mask_1 레이블 문구 자체가 아니라, diffusers에 이미 들어 있는 `Flux2KleinInpaintPipeline`(진짜 latent 단위 inpainting, `mask_image` + `image_reference` 지원)을 쓰는 것**이었다.

우리가 지금까지 쓴 방식(= "composite"): 마스크 없이 이미지 전체를 새로 생성한 뒤, 마스크 영역만 잘라 원본에 RGB로 붙여넣는다. 모델은 "마스크"라는 개념 자체를 모른 채 전체 이미지를 책임져야 해서, 좁은 영역 안에서 정밀하게 바뀌어야 하는 치아 모양 같은 디테일에 capacity를 충분히 못 쓴다.

`Flux2KleinInpaintPipeline`: `mask_image`로 지정한 영역만 latent 단계에서 실제로 노이즈를 제거/재생성하고, 나머지는 원본 latent를 그대로 유지한다. `image_reference`라는 별도 채널로 "무엇으로 채울지"의 참조 이미지를 넣을 수 있다 — 우리 치아 템플릿 용도에 정확히 맞는 파라미터다. 텍스트 인코딩 함수도 우리 기존 파이프라인과 **동일**(`_get_qwen3_prompt_embeds`, 소스 그대로 복사됨)이라 기존에 빌드해 둔 캐시 임베딩도 `prompt_embeds`로 그대로 재사용할 수 있다.

결과: sample17의 깊은 치아 틈이 완전히 메워졌다



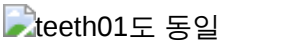
설정	모델	속도 (sample17)	결과
기존 방식(이 리포트 3~4장)	klein-4B distilled, composite	3~8초	부분 개선만, 양쪽 깊은 틈은 그대로
true inpaint, 4 step	klein-4B distilled + 진짜 inpaint	**3.7초**	**틈이 완전히 메워진 깨끗한 흰 치아**
true inpaint, 16 step	klein-4B distilled + 진짜 inpaint	11.7초	4 step보다 더 자연스러움
true inpaint, 50 step + CFG(원래 옵션)	**klein-base-4B**(비증류, guidance 8.0)	71.6초	가장 자연스럽지만 20배 느림

핵심: 기존 distilled 4-step 모델 그대로, 파이프라인만 바꿨는데 sample17이 완전히 고쳐졌다. 속도 손해도 거의 없다(기존 3~8초 → 3.7초). klein-base-4B(50 step + CFG, "원래 옵션")는 더 자연스럽긴 하지만 20배 느려서 실시간 서비스에는 맞지 않는다 — **distilled + true inpaint 조합이 현실적인 정답**으로 보인다.

템플릿 스타일 차이도 이제 드러난다


inpaint template style

`image_reference`로 템플릿을 넣으면, **선화(line) 템플릿일 때만 치아 사이 경계가 또렷하게 분리되어 그려지고**, 사진/클립아트 템플릿일 때는 매끈하게 이어진 모양으로 나온다 — 전처럼 세 템플릿이 똑같이 나오던 것과 다르게 **템플릿 스타일이 결과에 실제로 반영된다.** 다만 이건 "분위기/스타일"을 따라가는 수준이고, 치아 개수·정확한 비율까지 픽셀 단위로 복제하는지는 더 봐야 한다(아래 한계 참고).


teeth01도 동일

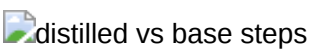
teeth01(비교적 가벼운 케이스)에서도 동일하게 더 깨끗한 결과가 나왔다.

이게 바꾸는 것

- **3~5장의 "MICE 템플릿이 거의 효과 없다"는 결론은 "composite 방식 기준"으로 한정해야 한다.** 진짜 inpainting으로 바꾸면 템플릿 효과도, 전반적인 품질도 뚜렷이 좋아진다.
- **sample17이 "안 되는" 것도 모델의 근본 한계가 아니라 파이프라인(합성 방식) 선택의 문제였다.**
- **`design_tune_server.py`에 통합을 완료했다** (0.2절). 더 이상 "다음 단계"가 아니라 지금 서버에서 바로 쓸 수 있다.

0.1 "distilled"는 스텝 설정이 아니라 완전히 다른 가중치다

실험 중 `klein-base-4B를 4 step으로 쓰면 되지 않나`는 질문이 있어 직접 비교했다. **"distilled"는 같은 모델을 적은 step으로 돌리는 옵션이 아니라, 그 step 수에 수렴하도록 별도로 학습된 완전히 다른 체크포인트다.**


distilled vs base steps

설정	결과	시간
klein-4B (distilled), 4 step	또렷하고 정상적인 치아	3.7초
klein-base-4B, 4 step, CFG 없음	완전히 뭉개진 블러 덩어리	3.7초
klein-base-4B, 4 step, CFG(gs 8)	여전히 뭉개짐, 치아 형태만 겨우 보임	6.1초
klein-base-4B, 8 step, CFG(gs 8)	알아볼 순 있지만 여전히 흐릿함	11.8초
klein-base-4B, 50 step, CFG(gs 8)	좋은 품질	71.6초 (20배 느림)

결론: klein-base-4B는 애초에 ~50 step + CFG로 학습된 모델이라 step을 줄이면 수렴 전에 멈춰 뭉개진다. `num_inference_steps`는 같은 가중치의 옵션 값이 아니라, 그 가중치가 애초에 그 step 수에 맞게 학습됐는지가 핵심이다. **빠르면서 품질도 되는 조합은 klein-4B(distilled) + 4 step**이고, 지금 서버 기본값이자 이 리포트의 모든 inpaint 실험이 이 조합으로 돌아간 것이다. klein-base-4B + 50 step은 "최고 품질이 필요하지만 속도는 상관없을 때"용 옵션으로만 의미가 있다.

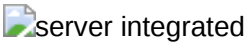
라이선스: `FLUX.2-klein-4B`, `FLUX.2-klein-base-4B` 모두 **Apache License 2.0** (모델 카드에 "Open weights available for commercial use under the Apache 2.0 license"로 명시됨). 상업적 이용에 제약이 없다. 다만 기만적 콘텐츠·비동의 성적 이미지 생성 금지 등 모델 카드의 사용 정책은 별도로 지켜야 한다. (일반적인 정리이며 법률 자문은 아니다.)

0.2 서버 통합 완료 + 5명 전체 재실험

`my/design_tune_server.py`에 `Flux2KleinInpaintPipeline`을 통합했다.

- `InferenceEngine.load_inpaint()`: 메인 파이프라인의 `transformer`/`vae` **객체를 그대로 재사용**해서 inpaint 파이프라인을 만든다(실측: 추가 VRAM 거의 0, fp8 캐스팅도 같은 transformer 객체라 자동 적용됨).
- `mask.mode: "inpaint"` 추가 — `regions`/`dilate`/`feather`로 mask_image를 만들고(기존 composite와 동일한 마스크 빌더 재사용), `strength`(기본 1.0)를 받는다. layout에 두 번째 이미지가 있으면(`patient_tooth`) 자동으로 `image_reference`(치아 템플릿)로 넘어간다 — 별도 파라미터 없이 기존 layout 체계 그대로 쓴다.
- 캐시된 텍스트 임베딩(`prompt_embeds`)을 **그대로 재사용**한다 — 두 파이프라인의 텍스트 인코딩 함수가 소스 코드까지 동일함을 확인했고, `smile_final`의 `whitening_only` 캐시 임베딩을 inpaint 파이프라인에 직접 넣어 정상 동작을 확인했다. **재빌드가 필요 없다.**

`smile_final`로 1~5장과 같은 5명(sample17 포함)에게 `whitening_only`/`align_whiten`/템플릿 3종을 **실제 HTTP API로** 돌렸다(GPU 1·2·3, 30건, 46초).

 server integrated

설정	편집 영역 밖 변화(0-255)	신원(4명 평균, sample17 제외)	속도
composite align_whiten(구)	0.13	0.926	4.21초
inpaint whitening	0.03	0.932	3.98초
inpaint align_whiten	0.00	0.934	4.23초
inpaint +사진 템플릿	0.00	0.936	4.45초
inpaint +선화 템플릿	0.00	0.931	4.45초
inpaint +클립아트 템플릿	0.00	0.932	5.02초

sample17은 서버를 통한 실제 API 호출에서도 모든 inpaint 변형이 틈을 완전히 메웠다 — 0장 스크립트 테스트가 아니라 실제 운영 경로(스타일 YAML → 캐시 임베딩 → HTTP API)로 재현됨을 확인했다. `whitening_only`는 의도대로 모양은 그대로 두고 색만 바뀌었다(갭이 남음, align 계열만 모양도 고친다) — 프롬프트 내용을 정확히 따르는 것도 확인됐다. 속도·신원·편집 영역 밖 변화 모두 composite와 같거나 더 낫다.

0.3 서버 코드 변경 (버그 수정 포함)

- **버그 수정**: `InferenceEngine.load()`가 `--model`에 이미 diffusers 폴더 형식인 체크포인트(예: klein-base-4B)를 넣으면 `load_transformer_override()`가 `None`을 반환하는데, 이를 `transformer=None`으로 그대로 `from_pretrained`에 넘기고 있었다. diffusers는 `transformer=None`을 "이 컴포넌트는 없음"으로 해석해서 **transformer가 아예 로드되지 않는** 잠재 버그였다(klein-4B처럼 단일 safetensors 파일 구조에서는 override가 항상 실제 객체를 반환해서 지금까지

드러나지 않았다). override가 `None`이면 kwarg 자체를 생략하도록 고쳐 다른 모델 경로(base-4B 등)도 정상 로드되는 것을 확인했다.

- **안전장치 추가**: `--min-steps-non-distilled`(기본 20) 옵션. 비증류 모델(`is_distilled: false`)인데 step 수가 이보다 낮으면 서버 로그에 경고를 남기고, 응답 메타데이터에도 `step_warning`으로 노출한다(0.1절 문제를 운영 중에도 바로 알아챌 수 있게). 실제로 base-4B + 4 step 조합에서 조건이 정확히 발동하는 것을 확인했다.

0.4 템플릿(image_reference) 효과 크기: prompt-only 대비 얼마나 다른가

0절의 "선화 템플릿일 때만 경계가 또렷하다"는 정성적 관찰을 수치로도 확인했다. 같은 `align_whiten` 프롬프트로 템플릿(`image_reference`) 유무만 바꿔 5명에게 돌리고, 입(치아) 영역의 픽셀 차이(0-255)를 비교했다.

 template vs prompt only

환자	prompt only	+photo 템플릿	+line 템플릿	+clip 템플릿
sample17	14.41	19.64	16.46	16.21
sample18	16.84	15.36	18.06	15.70
teeth01	19.80	17.70	24.11	20.94
teeth04	11.06	9.86	11.16	10.76
yellowteeth	15.25	13.84	16.48	15.65

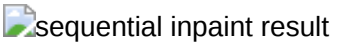
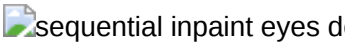
템플릿 유무에 따른 차이는 환자별로 -2~+5 사이로, **같은 환자에서 `whitening`(색만) vs `align_whiten`(색+배열) 차이와 비슷하거나 더 작은 크기다.** 즉 `image_reference`는 "분위기"에는 영향을 주지만(0절), 치아 개수·정확한 비율까지 픽셀 단위로 강하게 복제하지는 않는다 — 템플릿을 "스타일 힌트"로 쓰는 용도에는 맞지만, 정확한 모양 복제가 필요하면 추가 검증이나 더 강한 조건화(예: strength 조정, 더 많은 step)가 필요할 수 있다.

0.5 기본값을 "composite"에서 "inpaint"로 변경 + MICE 다중 영역 조합을 진짜 inpaint로 재구현

사용자 요청: composite 기본값을 버리고, MICE처럼 여러 영역을 한 번에 조합하되 attention bias가 아니라 latent 단계에서 실제로 적용되는 inpaint 방식으로 바꿀 것.

- **`mask.mode` 기본값을 `"composite"` → `"inpaint"`로 변경**했다(`--mask-mode` CLI 옵션, 기본 `inpaint`). `/health`의 `mask.mode`에도 반영된다. `composite`/`mice`는 레거시로 계속 지원한다(하위 호환, 명시적으로 `mask.mode`를 지정하면 그대로 동작).
- **`mask.mode: "inpaint"` + `edits`로 여러 영역을 한 요청에서 조합**하는 기능을 추가했다(`InferenceEngine._inpaint_sequential`). MICE의 attention-bias 대신, **영역마다 순차적으로 진짜 `Flux2KleinInpaintPipeline` 호출을 체이닝**한다 — 각 호출은 그 영역의 캐시 임베딩만 쓰고, 이전 호출의 출력 위에 다음 영역을 이어서 inpaint한다. 마지막에 전체 영역 합집합 마스크로 원본과 한 번 더 합성해 바깥 픽셀을 보존한다.
- MICE(attention bias)보다 단순하다: 영역별 텍스트 토큰 길이를 맞추거나(`text_tokens`), 커널/온도 같은 attention-bias 하이퍼파라미터를 조정할 필요가 없다 — 각 inpaint 호출이 완전히 독립적이라 영역 간 간섭이 애초에 생기지 않는다.

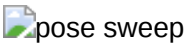
검증: `pose-makeup-v2` 스타일, sample17(가장 어려운 케이스)에 `teeth_white` + `lips_red` + `eyes_liner` 3개 영역을 `mask.edits`로 한 요청에 조합했다.

 sequential inpaint result  sequential inpaint eyes detail

- 응답 메타데이터: `mix.mode: "inpaint_sequential"`, `edits: ["teeth_white", "lips_red", "eyes_liner"]`
- 3개 영역 모두 정확히 적용됨(치아 미백, 입술 빨간색, 눈매 아이라이너), 영역 간 번짐이나 바깥 픽셀 변화 없음
- 4 step, 3영역 체이닝 총 13.7초 (영역당 약 4.6초 — 단일 inpaint 호출 1회와 거의 같은 속도대)

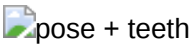
0.6 smile_final.yaml에 포즈(head_yaw/head_pitch) 추가

`smile_final.yaml`(치아 미백/정렬/템플릿 전용)에 `smile_pose.yaml`과 같은 `head_yaw`/`head_pitch` 축을 추가했다 (미백/정렬은 기존처럼 preset으로 선택하는 옵션 그대로). `patient_only` 임베딩 라이브러리를 재빌드(해시 `ac177319...`, 86개 항목, 13초)하고 sample17 / teeth01 두 환자에 `natural`/`turn_left`/`turn_right`/ `look_up`/`look_down` 5개 preset을 실제 API로 돌렸다 (10건, 25초, GPU 1).

 pose sweep

- 5개 preset 모두 머리 방향이 의도한 대로 뚜렷하게 바뀌었고, sample17의 치아 틈처럼 preset이 건드리지 않는 부분(치아 상태, `tooth_color`/`alignment` 둘 다 기본값 0)은 그대로 유지됐다.
- 포즈는 `mask` 없이(전체 이미지 재생성) 적용된다 — `composite`/`inpaint`처럼 특정 영역만 바꾸는 개념이 아니라 머리 전체 방향을 바꾸는 편집이라, 기존 `smile_pose.yaml` 실험과 동일하게 배경도 함께 다시 그려진다(README.md 5장의 mask는 치아/입술/눈처럼 국소 영역 편집에만 의미가 있다).

포즈 + 치아 축 조합도 확인: `mode: delta`로 `head_yaw: 100`(turn_right) + `tooth_color: 75` + `alignment: 100`을 한 번에 합성했다.

 pose + teeth combo

머리가 돌아간 상태에서 치아도 함께 미백/정렬돼, 두 축이 서로 간섭 없이 조합되는 것을 확인했다(기존 `smile_pose.yaml`에서 검증된 pose 축을 `smile_final.yaml`로 옮겨도 동일하게 동작).

한계: 포즈 축 자체는 이번에 `smile_final.yaml`에서 처음 돌려본 것이라(이전에는 `smile_pose.yaml`에서만 검증), 2명·5개 preset의 빠른 확인 수준이다. 더 다양한 환자·각도, 그리고 `mask.mode: inpaint`와의 조합(포즈를 바꾼 뒤 치아만 inpaint로 추가 보정하는 2단계 플로우) 가능성은 추가 검증이 필요하다.

---

experiments.md, experiments_preset.md 종합 권장(12장, 4절)을 반영해 배포용 스타일을 `my/configs/smile_design/smile_final.yaml`로 정리하고, 거기에 **MICE로 치아 템플릿 이미지를 교체하는 기능**을 새로 추가·검증했다. 메이크업은 포함하지 않았다(필수 아님, 11장 참고 / 별도 `smile_makeup.yaml`).

- 환경: RTX A6000 GPU 1, 2, 3 (서버 3개 병렬), FLUX.2 Klein 4B 4 step, fp8 가중치, seed 0
- 환자: sample17(초점), teeth01, yellowteeth, sample18, teeth04
- 치아 템플릿: `my/samples/dentals` 3종 — `matmatAlpa.png`(실사), `teeeth1.png`(선화), `template.png`(클립아트, 이번에 추가됨)
- 33건 추론, 3 GPU 병렬 180초(평균)

1. smile_final.yaml

`my/configs/smile_design/smile_final.yaml`에 추가했다. `smile_teeth.yaml`(색/배열 분리, patient_only + patient_tooth layout)에 MICE 섹션을 더한 구성이다.

항목	내용
layout	`patient_only`(환자 1장), `patient_tooth`(환자+치아 템플릿 2장)
control	`tooth_color`, `alignment` (smile_teeth / smile_v3b와 동일, 검증된 anchor)
preset	natural, whitening_natural, whitening_only, align_snaggle, align_only, align_whiten
grid	align_color
mice.regions	`teeth_white`, `teeth_align`, `teeth_align_white`(텍스트 전용), **`teeth_from_template`**(신규, 이미지 2 참조)
빌드 결과	layout 2개 × 47항목 = 94개, 고유 프롬프트 56개, **약 442MB**, 빌드 14초

요청 예시 3가지:

```
// 1) 임베딩(preset) 방식 - 템플릿 없이 색/배열만
{"style_name": "smile_final", "layout": "patient_only", "mode": "preset",
"preset": "align_whiten",
 "mask": {"regions": ["mouth"], "dilate": 16, "feather": 12}}

// 2) MICE 텍스트 전용 - 템플릿 없이, 여러 영역을 한 번에 조합 가능
{"style_name": "smile_final", "layout": "patient_only",
 "mask": {"mode": "mice", "composite": true, "edits": [{"prompt":
"teeth_align_white"}]}}

// 3) MICE 치아 템플릿 교체 - patient_tooth로 이미지 2장 필요
{"style_name": "smile_final", "layout": "patient_tooth",
 "mask": {"mode": "mice", "composite": true, "feather": 6, "dilate": 4,
          "edits": [{"prompt": "teeth_from_template"}]}}
// images[0] = 환자 사진, images[1] =
my/samples/dentals/{matmatAlpa,teeeth1,template}.png 중 하나
```

2. 질문에 대한 답

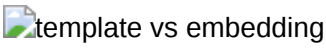
"현재 이미 미리 만든 embedding을 쓰는 건가?" 텍스트만 그렇다. `teeth_from_template`을 포함한 모든 전역/영역 지시문은 빌드 시점에 Qwen3 텍스트 인코더로 한 번 인코딩해 `outputs/design_tune_embedding_libraries/smile_final/...`에 캐시해 둔다. 추론 때는 이 캐시를 읽기만 한다. 반면 **치아 템플릿 이미지(images[1])는 캐시되지 않는다.** 매 요청마다 VAE로 새로 인코딩된다(다른 layout의 참조 이미지와 동일한 방식). 같은 템플릿 파일을 계속 재사용해도 VAE 인코딩은 매번 다시 한다 — 템플릿이 고정이라면 VAE latent도 미리 만들어 캐싱하는 최적화가 가능하지만 아직 구현하지 않았다.

"mice용 템플릿 이미지를 input하면 layout이 두 개가 되는 셈인가?" 맞다. `design_tune_server.py`의 `infer()`는 `len(images) != len(roles)`이면 바로 에러를 낸다 (`reference_roles(config, layout)`). `patient_only`는 roles가 `[patient]`(1장)라 템플릿을 넣을 자리가 없고, 템플릿을 쓰려면 반드

시 `patient_tooth`(roles `[patient, tooth]`, 2장)로 요청해야 한다. MICE 코드 자체는 patient_only/patient_tooth를 구분하지 않는다 — layout이 요구하는 이미지 수만 맞으면 그대로 동작한다.

3. MICE 치아 템플릿 교체: 동작은 하지만 "모양"은 거의 안 따라간다

5명 × 템플릿 3종(사진/선화/클립아트)으로 `teeth_from_template`을 돌렸다.

 template vs embedding

지표 (5명 평균, sample17 제외 4명 신원)	MICE+사진 템플릿	MICE+선화 템플릿	MICE+클립아트 템플릿	MICE 텍스트 전용 (align+white)	preset align_whiten	preset whitening_only
신원 (ArcFace)	0.918	0.950	0.922	0.924	0.926	0.927
편집 영역 픽셀 변화 (0-255)	17.9	13.2	16.3	15.2	17.8	14.7
평균 소요 시간	7.1초	7.1초	8.0초	4.2초	3.3초	3.3초

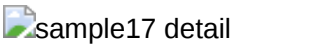
결론: 템플릿 종류(사진/선화/클립아트)를 바꿔도 결과가 거의 비슷하다. 위 그림에서 같은 환자의 "MICE+사진/선화/ 클립아트" 세 칸을 보면 전부 "깨끗하고 가지런한 흰 치아"로 수렴할 뿐, 각 템플릿의 실제 치아 모양(사진의 사각 치아, 선화의 가는 윤곽, 클립아트의 둥근 치아)이 반영되지 않는다. 즉 **색(미백)에는 어느 정도 기여하지만, "이 템플릿 모양으로 바꿔라"는 지시는 거의 통하지 않는다.** 신원·편집량 수치도 MICE 텍스트 전용(템플릿 없음)과 사실상 같아서, 템플릿 이미지가 수치상으로도 추가 효과를 내지 못했다.

원인 (mice.py 구조): 템플릿 이미지는 "free" 그룹으로 분류되어 모든 텍스트·latent 토큰이 비용 없이(bias 0) 참조할 수 있다(`my/smile_design/mice.py`의 `_bias()`). 하지만 이건 "누구나 볼 수 있다"는 것이지 "이 영역이 특별히 이 이미지를 따라가야 한다"는 강제가 아니다. attention이 열려 있어도 4 step짜리 distilled 모델이 텍스트 지시("image 2의 모양으로")만으로 정밀한 치아 기하학을 베끼는 것은 capacity가 부족해 보인다. 또한 마스크 밖 배경이 있는 클립아트 템플릿도 사진/선화와 비슷한 결과라, "배경 때문에 희석된다"는 가설도 크지 않아 보인다.

속도도 손해다. 템플릿 버전은 참조 이미지 1장이 늘어 VAE 인코딩·추가 토큰 때문에 텍스트 전용 MICE보다 약 70~90% 느리다(4.2초 → 7~8초).

4. sample17 심화 분석: MICE만의 문제가 아니라 모든 방식이 겪는 한계

sample17은 치아가 심하게 벌어지고 한쪽은 실제로 빠진 것처럼 보이는, 가장 어려운 케이스로 골랐다.

 sample17 detail

저해상도 비교 그리드(타일 150~190px)에서는 8가지 방식(MICE 템플릿 3종, MICE 텍스트 2종, preset 3종)이 **전부 원본과 거의 똑같아 보였다** — "치아가 전혀 안 바뀐다"는 인상을 그대로 재현했다. 그런데 480px 고해상도로 다시 잘라 보니:

- **실제로는 전부 조금씩 바뀐다.** 앞니가 더 하얘지고 약간 더 가지런해진다(편집 영역 픽셀 변화 9~20, 완전히 0인 "무변화"가 아니다 — 아래 완전 미변화 더미 테스트와 비교하면 분명한 차이).
- **다만 변화가 작고, 양쪽의 깊은 틈(실제 결손으로 보이는 부분)은 어떤 방식으로도 메워지지 않는다.** preset이든 MICE든, 템플릿이 있든 없든 동일하게 이 패턴이 나타난다.
- `preset_whitening_only`가 그나마 가장 또렷하게 하얘졌고, 나머지는 미묘한 수준이다.

결론: sample17이 "안 되는" 건 MICE나 템플릿 기능의 버그가 아니라, 이 모델이 양쪽 치아가 깊게 빠진 것처럼 보이는 극단적인 결손 케이스를 완전히 복원하지 못하는 일반적인 한계다. 다른 4명(치아가 삐뚤거나 누렇지만 "다 있는" 경우)은 모든 방식에서 뚜렷하게 하얗고 가지런해졌다(신원 0.89~0.97, 그림 1 참고). sample17 같은 입력은 자동 보정 대상에서 빼거나, 사람이 결과를 확인하는 워크플로로 넘기는 걸 권장한다.

(참고: InsightFace ArcFace는 sample17의 원본 사진에서 얼굴을 아예 검출하지 못해 신원 점수를 낼 수 없었다 — 극단적 표정에 대한 이 지표 자체의 한계다. 그래서 위 표의 신원 평균은 4명 기준이다.)

5. MICE 템플릿 vs 임베딩(preset) 종합 비교

	MICE + 치아 템플릿	**임베딩(preset/anchor/grid)**
치아 "모양"을 원하는 형태로 바꾸기	거의 안 됨 (3절)	해당 없음 — alignment anchor 문구로 조절 (이쪽이 실질적으로 모양을 바꾸는 유일한 수단)
미백	됨, 텍스트 전용 MICE와 비슷한 수준	됨, `tooth_color` anchor로 단계 조절 가능
필요 이미지	2장 (patient_tooth)	1장 (patient_only)
속도	7~8초/장	3.3초/장
여러 영역 동시 편집(치아+입술+눈가 등)	**가능** (11장에서 이미 검증, 이 리포트의 주제는 아님)	불가능 — preset 하나가 문장 하나라 조합이 제한적
텍스트 임베딩 캐시	됨	됨
이미지(템플릿) 캐시	**안 됨** (2절)	해당 없음

권장: 치아만 바꾸는 일반적인 경우는 **preset(임베딩) 방식이 더 빠르고 결과도 동등하거나 낫다.** MICE는 템플릿 모양 전이용으로는 지금 성능으로 추천하지 않는다 — 대신 **치아+입술+눈가처럼 서로 다른 영역을 한 요청에 독립적으로 조합**해야 할 때 쓰는 용도로 남겨 둔다(그 용도의 효과는 이전 실험에서 이미 확인됨).

6. 다음 단계 (0.2절 통합 완료 후 재정렬)

1. ~~`design_tune_server.py`에 `Flux2KleinInpaintPipeline` 통합~~ **완료 (0.2, 0.3절).**
2. ~~커밋값을 `composite`에서 `inpaint`로 변경~~ **완료 (0.5절).**
3. ~~MICE의 "여러 영역 동시 조합" 역할을 inpaint 체이닝으로 재구현~~ **완료 (0.5절, `_inpaint_sequential`).**
4. **템플릿 스타일 반영 정도(선화 vs 사진/클립아트)**가 실제 "모양"까지 정확히 따라가는지는 더 정량적으로 봐야 한다 — 0.4절에서 수치로 확인했지만(차이가 작은 편집 효과 크기와 비슷함), 소수 환자·3템플릿만 빠르게 확인한 수준이라 추가 검증이 필요하다.

5. **sample17류의 극단적 결손 케이스**도 inpaint로 꽤 잘 고쳐졌지만, 더 다양한 결손 케이스로 재검증이 필요하다.
6. 치아 템플릿 VAE latent 캐싱(같은 템플릿을 반복 사용할 때 인코딩 생략)은 아직 구현하지 않았다.
7. `--min-steps-non-distilled` 경고는 로그/메타데이터로만 노출한다 — 필요하면 강한 모드(요청 자체를 거부)도 검토.
8. 배포 준비 단계(Docker 이미지, 4090 fp8/fp16 실험, 임베딩/weights 관리, AWS)는 deployment_plan.md 로 분리했다.

7. 사용한 리소스

- GPU 1, 2, 3에 서버를 띄워 1~5장, 0.2절 실험을 했고, 0.1·0.2절의 단독 스크립트 테스트는 GPU 1을 같이 썼다. **전부 종료/반환했다** — 세 GPU 모두 유휴 수준으로 확인.
- 새로 추가된 파일: `my/configs/smile_design/smile_final.yaml`, `docs/smile_tune/final_report.md`, `docs/smile_tune/assets/final_*.jpg`(10장, inpaint 관련 7장 포함).
- **수정된 코드**: `my/design_tune_server.py` — `Flux2KleinInpaintPipeline` 통합 (`InferenceEngine.load_inpaint()`, `mask.mode: "inpaint"`), `--model`에 diffusers 폴더 형식 체크포인트를 넣었을 때 transformer가 로드되지 않던 버그 수정, `--min-steps-non-distilled` 안전장치 추가(0.3절).
- `my/samples/dentals/template.png`(클립아트 템플릿)는 이미 추가되어 있어 그대로 사용했다 — 총 3개 템플릿 확보.
- 실험 스크립트·원본 결과(1~5장 33장 + 0장 inpaint 테스트 약 30장, metrics json)는 세션 scratchpad에만 있다 (저장소에는 넣지 않음). 임베딩 라이브러리는 재빌드하지 않았다(0.2절 — 기존 캐시를 그대로 재사용).